

5.5. Bankruptcy in Taiwan tw

```
In [1]: import gzip
import json
import pickle

import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from imblearn.over_sampling import RandomOverSampler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    classification_report,
    confusion_matrix,
)
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_sp
```

Prepare Data

Import

Task 5.5.1

```
In [2]: with gzip.open("data/taiwan-bankruptcy-data.json.gz", "r") as f:
        taiwan_data = json.load(f)

print(type(taiwan_data))
```

<class 'dict'>

Task 5.5.2

Tip: The data in this assignment might be organized differently than the data from the project, so be sure to inspect it first.

```
In [3]: taiwan_data_keys = taiwan_data.keys()
print(taiwan_data_keys)

dict_keys(['schema', 'metadata', 'observations'])
```

Task 5.5.3

```
In [4]: n_companies = len(taiwan_data["observations"])
print(n_companies)
```

6137

Task 5.5.4

```
In [5]: n_features = len(taiwan_data["observations"][0])
print(n_features)
```

97

Task 5.5.5

```
In [6]: def wrangle(filename):
# Open compressed file and load contents
with gzip.open(filename, "r") as f:
    data = json.load(f)

# Turn dict into DataFrame
df = pd.DataFrame().from_dict(data["observations"]).set_index("id")

return df

df = wrangle("data/taiwan-bankruptcy-data.json.gz")
print("df shape:", df.shape)
df.head()
```

df shape: (6137, 96)

```
Out[6]:
```

	bankrupt	feat_1	feat_2	feat_3	feat_4	feat_5	feat_6	feat_7	feat_8
id									
1	True	0.370594	0.424389	0.405750	0.601457	0.601457	0.998969	0.796887	0.800000
2	True	0.464291	0.538214	0.516730	0.610235	0.610235	0.998946	0.797380	0.800000
3	True	0.426071	0.499019	0.472295	0.601450	0.601364	0.998857	0.796403	0.800000
4	True	0.399844	0.451265	0.457733	0.583541	0.583541	0.998700	0.796967	0.800000
5	True	0.465022	0.538432	0.522298	0.598783	0.598783	0.998973	0.797366	0.800000

5 rows × 96 columns



Explore

Task 5.5.6

```
In [7]: nans_by_col = df.isnull().sum()
print("nans_by_col shape:", nans_by_col.shape)
nans_by_col.head()
```

nans_by_col shape: (96,)

```
Out[7]: bankrupt    0
feat_1             0
feat_2             0
feat_3             0
feat_4             0
dtype: int64
```

Slight Code Change

In the following task, you'll notice a small change in how plots are created compared to what you saw in the lessons. While the lessons use the global matplotlib method like `plt.plot(...)`, in this task, you are expected to use the object-oriented (OOP) API instead. This means creating your plots using `fig, ax = plt.subplots()` and then calling plotting methods on the `ax` object, such as `ax.plot(...)`, `ax.hist(...)`, or `ax.scatter(...)`.

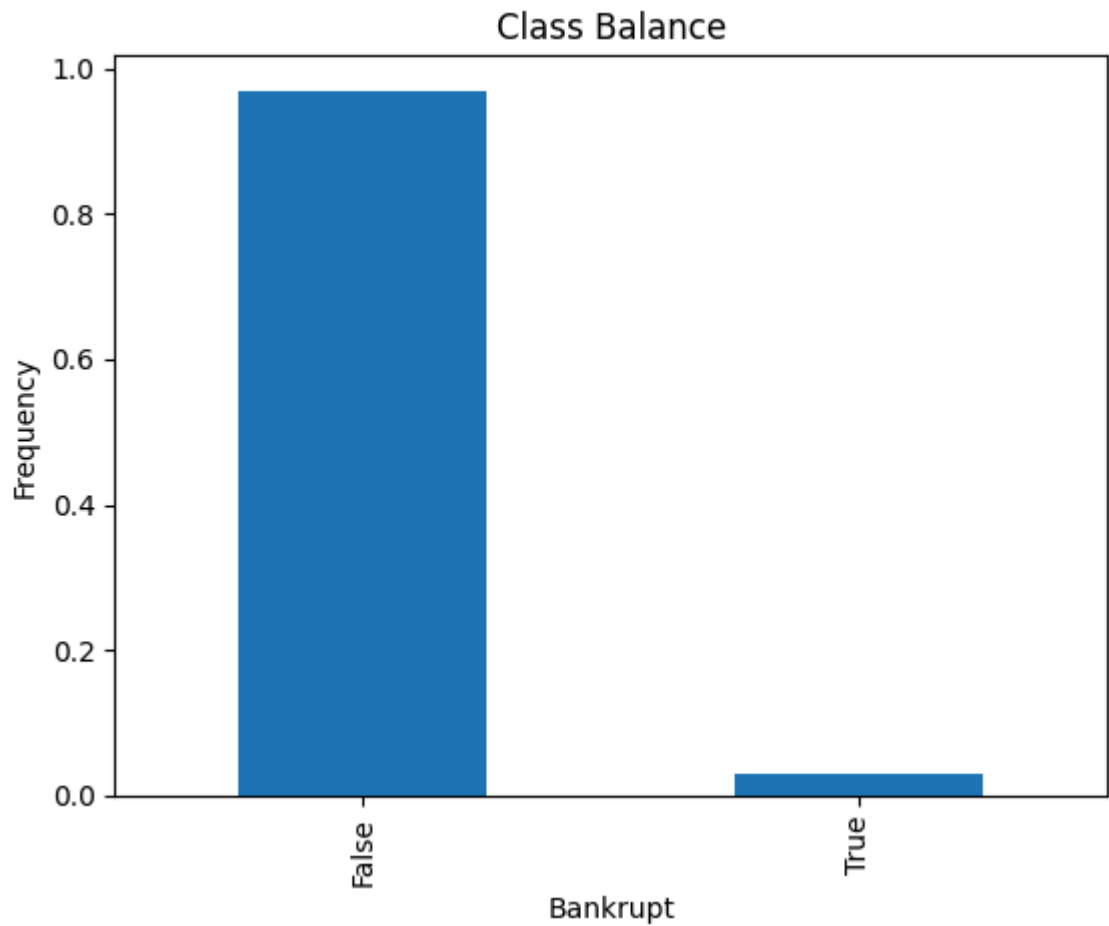
If you're using pandas' or seaborn's built-in plotting methods (like `df.plot()` or `sns.lineplot()`), make sure to pass the `ax=ax` argument so that the plot is rendered on the correct axes.

This approach is considered best practice and will be used consistently across all graded tasks that involve matplotlib.

Task 5.5.7

```
In [8]: # Plot class balance
fig, ax = plt.subplots()
df["bankrupt"].value_counts(normalize=True).plot(
    kind="bar",
    xlabel="Bankrupt",
    ylabel="Frequency",
    title="Class Balance",
    ax=ax,
)
```

```
Out[8]: <Axes: title={'center': 'Class Balance'}, xlabel='Bankrupt', ylabel='Frequency'>
```



Split

Task 5.5.8

```
In [9]: target = "bankrupt"
X = df.drop(columns=target)
y = df[target]
print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (6137, 95)
y shape: (6137,)
```

Task 5.5.9

```
In [10]: X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (4909, 95)
y_train shape: (4909,)
X_test shape: (1228, 95)
y_test shape: (1228,)
```

Resample

Task 5.5.10

```
In [11]: over_sampler = RandomOverSampler(random_state=42)
X_train_over, y_train_over = over_sampler.fit_resample(X_train, y_train)
print("X_train_over shape:", X_train_over.shape)
X_train_over.head()
```

X_train_over shape: (9512, 95)

```
Out[11]:
```

	feat_1	feat_2	feat_3	feat_4	feat_5	feat_6	feat_7	feat_8	feat_9
0	0.535855	0.599160	0.594411	0.627099	0.627099	0.999220	0.797686	0.809591	0.303
1	0.554136	0.612734	0.595000	0.607388	0.607388	0.999120	0.797614	0.809483	0.303
2	0.549554	0.603467	0.599122	0.620166	0.620166	0.999119	0.797569	0.809470	0.303
3	0.543801	0.603249	0.606992	0.622515	0.622515	0.999259	0.797728	0.809649	0.303
4	0.498659	0.562364	0.546978	0.603670	0.603670	0.998904	0.797584	0.809459	0.304

5 rows × 95 columns



Build Model

Iterate

Task 5.5.11

```
In [12]: clf = RandomForestClassifier(random_state=42)
clf
```

```
Out[12]:
```

▼

RandomForestClassifier

RandomForestClassifier(random_state=42)

Task 5.5.12

Tip: Use your CV scores to evaluate different classifiers. Choose the one that gives you the best scores.

```
In [13]: cv_scores = cross_val_score(
            clf, X_train_over, y_train_over, cv=5, n_jobs=-1
        )
print(cv_scores)
```

[0.99316868 0.99474514 0.99369085 0.99369085 0.9957939]

```
In [14]: # Run this code cell
params = {
    "max_depth": range(30, 50, 10),
```

```
"n_estimators": range(25, 51, 25),
}
params
```

```
Out[14]: {'max_depth': range(30, 50, 10), 'n_estimators': range(25, 51, 25)}
```

Task 5.5.13

```
In [15]: model = GridSearchCV(
            clf,
            param_grid=params,
            cv=5,
            n_jobs=-1,
            verbose=1,
        )
model
```

```
Out[15]: ▸ GridSearchCV
          ▸ estimator: RandomForestClassifier
            ▸ RandomForestClassifier
```

Ungraded Task: Fit your model to the over-sampled training data.

```
In [16]: model.fit(X_train_over, y_train_over)
```

Fitting 5 folds for each of 4 candidates, totalling 20 fits

```
Out[16]: ▸ GridSearchCV
          ▸ estimator: RandomForestClassifier
            ▸ RandomForestClassifier
```

Task 5.5.14

```
In [17]: cv_results = pd.DataFrame(model.cv_results_)
cv_results.head(5)
```

Out[17]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_max_depth	param
--	---------------	--------------	-----------------	----------------	-----------------	-------

0	0.744927	0.062211	0.008089	0.001400	30	
---	----------	----------	----------	----------	----	--

1	1.344510	0.022765	0.011913	0.000426	30	
---	----------	----------	----------	----------	----	--

2	0.685901	0.016397	0.007750	0.001082	40	
---	----------	----------	----------	----------	----	--

3	1.356351	0.015160	0.011955	0.000255	40	
---	----------	----------	----------	----------	----	--



Task 5.5.15

```
In [18]: best_params = model.best_params_  
print(best_params)
```

```
{'max_depth': 40, 'n_estimators': 50}
```

Evaluate

Ungraded Task: Test the quality of your model by calculating accuracy scores for the training and test data.

```
In [20]: acc_train = model.score(X_train_over, y_train_over)  
acc_test = model.score(X_test, y_test)  
  
print("Model Training Accuracy:", round(acc_train, 4))  
print("Model Test Accuracy:", round(acc_test, 4))
```

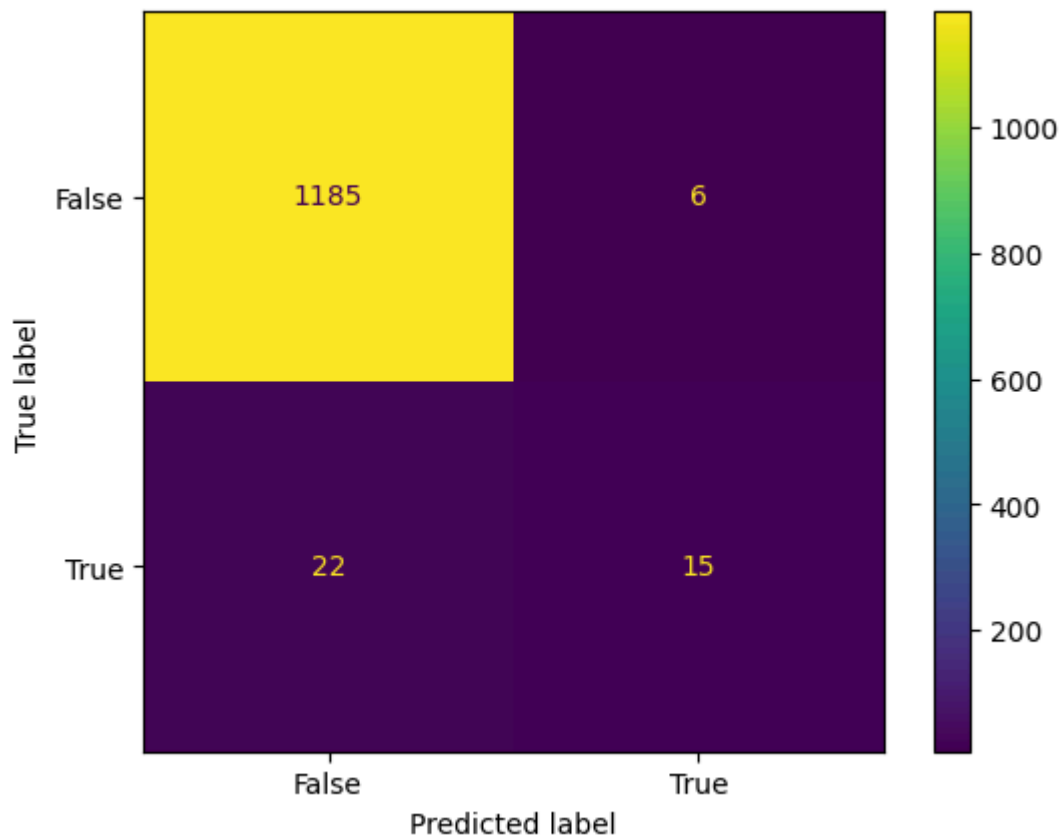
```
Model Training Accuracy: 1.0
```

```
Model Test Accuracy: 0.9772
```

Task 5.5.16

```
In [21]: # write code here  
fig, ax = plt.subplots()  
ConfusionMatrixDisplay.from_estimator(model, X_test, y_test, ax=ax)
```

```
Out[21]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x72c72fceb5  
0>
```



Task 5.5.17

```
In [22]: class_report = classification_report(y_test, model.predict(X_test))
print(class_report)
```

	precision	recall	f1-score	support
False	0.98	0.99	0.99	1191
True	0.71	0.41	0.52	37
accuracy			0.98	1228
macro avg	0.85	0.70	0.75	1228
weighted avg	0.97	0.98	0.97	1228

Communicate

Task 5.5.18

```
In [23]: # write your code here
# Extract importances from the best estimator
importances = model.best_estimator_.feature_importances_
feat_imp = pd.Series(importances, index=X_train_over.columns).sort_values()

# 10 most important features
feat_imp = feat_imp.tail(10)

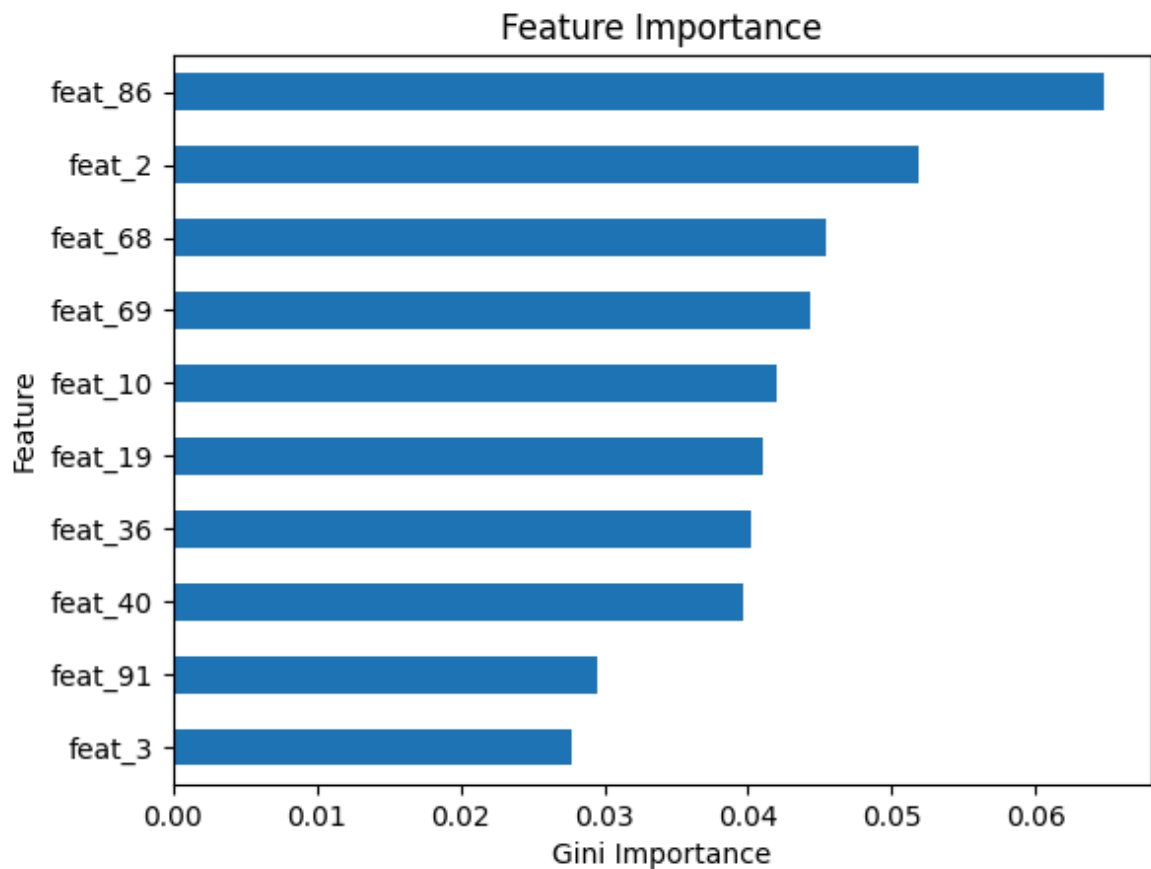
fig, ax = plt.subplots()
feat_imp.plot(
    kind="barh",
    xlabel="Gini Importance",
```

```

    ylabel="Feature",
    title="Feature Importance",
    ax=ax,
)

```

Out[23]: <Axes: title={'center': 'Feature Importance'}, xlabel='Gini Importance', ylabel='Feature'>



Task 5.5.19

```

In [24]: # Save model
with open("model-5-5.pkl", "wb") as f:
    pickle.dump(model, f)

```

Task 5.5.20

```

In [25]: # Import your module
from my_predictor_assignment import make_predictions

# Generate predictions
y_test_pred = make_predictions(
    data_filepath="data/taiwan-bankruptcy-data-test-features.json.gz",
    model_filepath="model-5-5.pkl",
)
print("predictions shape:", y_test_pred.shape)
y_test_pred.head()

```

predictions shape: (682,)

```
Out[25]: id
18      False
20      False
24      False
32      False
38      False
Name: bankrupt, dtype: bool
```

Tip: If you get an `ImportError` when you try to import `make_predictions` from `my_predictor_assignment`, try restarting your kernel. Go to the **Kernel** menu and click on **Restart Kernel and Clear All Outputs**. Then rerun just the cell above. 🙌

Copyright 2023 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.